



Save Custom Game

Documentation

Creator: Lucas Gomes Cecchini

Pseudonym: AGAMENOM

Overview

This document will help you use **Assets Save Custom Game** for **Unity**.

With it you can save your game in several Slots.

It will take a **screenshot** of the game and there are three ways to save in **GameData(.json)**, **LocalLow(.json)** and **PlayerPrefs**.

It also has Auto Save features based on **Time** or **Events**.

Instructions

You can get more information from the Playlist on YouTube:

<https://youtube.com/playlist?list=PL5hnfx09yM4JDFFdX1vfpNCLLdLq4ixZA&si=3w1IN0WrpV8IFMrS>

Script Explanations

Class

ExceptionUtility

This script is a utility meant to assist with debugging in a system called SaveCustomGame. It is designed to help developers track the **exact file and line number** where a certain method or event originated.

Purpose

The ExceptionUtility class provides a static method that returns a string containing the file path and line number of the *external* method that called it. This is useful for adding to error logs or debug messages so developers know exactly **where** in the code something happened — such as where a save or load operation was triggered.

How It Works

Class: ExceptionUtility

- It's a **static class**, which means it cannot be instantiated. All of its members are accessed directly through the class itself.
- It lives under the namespace SaveCustomGame, so it's part of the game saving system.

Method: GetCallingMethodInfo()

What it does:

- It captures the current **stack trace**, which is like a breadcrumb trail showing how the program reached the current line of code.
- It filters out internal calls from helper classes (specifically SaveDataUtility and ExceptionUtility) so you get only the *external* call that triggered the action.
- It then finds and returns the path and line number of that relevant call, formatted like (at Assets/Scripts/MyScript.cs:42).

Variables and Logic

stackTrace

- This variable holds the entire call history (stack trace) up to the current point in execution.
- It includes file names and line numbers because it's created with true.

frames

- This is a list of all the **frames** (steps) in the stack trace.
- Each frame represents a method call.

foundSaveDataUtility

- A flag (true/false) that keeps track of whether a method from SaveDataUtility has been found in the stack.
- This is important because the method stops analyzing frames if it sees that both SaveDataUtility and then ExceptionUtility have been processed.

Frame Filtering Logic

1. Loops through all frames in the stack.
2. Checks the class where each method came from.
3. If the method is **not from SaveDataUtility or ExceptionUtility**, it's considered an external call.
4. If the frame has both:
 - A file name (the script path), and
 - A line number (where in the script the method is),

it formats this into a string and returns it.

5. If it does see a frame from SaveDataUtility, it flips the foundSaveDataUtility flag.
6. If it then encounters a frame from ExceptionUtility **after** SaveDataUtility, it stops — assuming there's nothing else useful to find.
7. If no usable frame is found, it returns an **empty string**.

Typical Usage

Let's say you're writing save system code like this:

```
Debug.Log("Saving game started " + ExceptionUtility.GetCallingMethodInfo());
```

In your Unity console, this would output something like:

Saving game started (at Assets/Scripts/SaveGameManager.cs:88)

This helps you track down exactly **where in your project** this log was triggered, without manually tracing function calls.



Summary

- This utility is for **debugging and logging**, helping trace events to their origins.
- It skips helper utility calls to give you the *most relevant* line of code in your own scripts.
- Great for identifying where errors or unexpected behaviors start during save/load operations.

SaveDataUtility

Here's a full explanation of how the SaveDataUtility script works, including its variables, methods, and typical usage. This utility is part of a custom save system in Unity designed to handle multiple types of data (floats, ints, vectors, strings, bools) and even screenshots through a centralized asset called SaveCustomObject.

General Purpose

The SaveDataUtility class provides static helper methods to:

- Load data from a custom save asset (SaveCustomObject).
- Retrieve and update various types of values (Vector, Float, Int, String, Bool).
- Handle screenshots.
- Manage autosave behavior.
- Trigger manual save events.

Core Concepts

- **SaveCustomObject**: A Unity asset (ScriptableObject) that stores a list of SaveCustomItem objects.
- **SaveCustomItem**: Represents an item group identified by a unique tag, holding collections of data like floats, ints, vectors, etc.
- **SaveCustomVector, SaveCustomFloat, etc.**: Structures representing a tag-value pair for a specific data type.
- **[Save Custom Object]**: A scene object which may contain runtime data and components for saving during gameplay.

Methods Overview

GetSaveCustomObject

- Loads the asset Save Custom Object Data from the Resources folder.
- Returns it for access to save data.
- Logs an error if the object is not found.

GetVector / GetFloat / GetInt / GetString / GetBool

- Used to retrieve values from the save system based on itemTag (group) and a specific tag (vectorTag, floatTag, etc.).
- Searches through the save object to find the corresponding tag and return the value.
- If the tag or item is not found, a KeyNotFoundException is thrown (and the error is logged).

SetVector / SetFloat / SetInt / SetString / SetBool

- Sets a value in the save system.
- Searches for the corresponding itemTag and tag, then updates the value.
- If the tag doesn't exist, it creates and adds a new entry.
- SetVector is overloaded to accept multiple types (Vector2, Vector3, Vector4, Quaternion) and internally converts them into a Vector4.

CaptureScreenshot

- Captures a screenshot using a specific camera.
- Resizes the screenshot according to a pixelLimit stored in SaveCustomObject.

- Renders the camera's output to a RenderTexture, extracts it into a Texture2D, and stores the PNG-encoded byte array into the save object.

RenderScreenshot

- Takes the screenshot byte array and recreates a Texture2D from it.

TextureToSprite

- Converts a Texture2D into a Sprite, allowing it to be used in UI elements.

GetComponentSaveCustomInScene

- Looks for the scene GameObject named [Save Custom Object].
- Tries to retrieve a component of type SaveCustomInScene from it.
- Logs success or failure accordingly.

EnableAutoSave / DisableAutoSave

- Sets the autosaveEnabled flag inside the SaveCustomObject.
- This flag can be used to control whether automatic saving is allowed.

SaveEvent

- Finds the [Save Custom Object] in the scene.
- Looks for an AutoSaveCustom component and calls its SaveAutoGame method.
- Useful for triggering a save manually, outside of autosave.

Usage Example (Not in C# syntax)

1. **Reading data:**
 - Get the player's last saved health: GetFloat("PlayerStats", "Health")
 - Get saved position: GetVector("PlayerTransform", "Position")
2. **Writing data:**
 - Save new health: SetFloat("PlayerStats", "Health", 95.0)
 - Save rotation: SetVector("PlayerTransform", "Rotation", Quaternion object)
3. **Taking a screenshot:**
 - CaptureScreenshot(camera)
4. **Showing a screenshot in UI:**
 - RenderScreenshot(savedByteArray) → Texture2D
 - TextureToSprite(texture) → Sprite
5. **Autosave management:**
 - Enable: EnableAutoSave()
 - Disable: DisableAutoSave()
6. **Trigger a save:**
 - SaveEvent()

Final Notes

- The script assumes a well-defined structure in the SaveCustomObject, and tagging consistency is key.
- The method GetCallingMethodInfo() (not defined in this snippet) likely returns the method name for detailed error logging.
- Helpful for both runtime data persistence and developer debugging.

Components

AutoSaveCustom

This script is a Unity component that performs **automatic background saving** of game data at fixed time intervals. It's designed to make sure the game progress is saved regularly without requiring player interaction, and it cycles through a series of save slots to avoid overwriting the same file each time.

Purpose

The component's goal is to **automate the save process** using Unity's FixedUpdate() cycle. It saves the game periodically using data from another component (SaveCustomInScene) and obeys configuration options set in a SaveCustomObject, such as whether auto-saving is enabled and the time interval between saves.

Class: AutoSaveCustom

This is a MonoBehaviour script, meaning it can be attached to a GameObject in the Unity scene.

Variables and Their Roles

saveCustomInScene

- A reference to another component in the scene responsible for managing saving operations.
- If not assigned manually, it tries to find and assign itself via a utility.

currentAutoSaveSlot

- Tracks which of the save slots is currently in use.
- Starts from 1 and cycles through slots 1 to 6.

timeSinceLastSave

- Measures how much time (in seconds) has passed since the last autosave.
- Resets each time a save is performed.

saveInterval

- The amount of time to wait between each autosave.
- Pulled from the game's save settings (SaveCustomObject) at the beginning.

saveKey

- A string used as a key to store and retrieve the current autosave slot from Unity's PlayerPrefs system, ensuring continuity across game sessions.

How the Script Runs

1. Initialization – Start()

When the game starts:

- If saveCustomInScene hasn't been assigned, it fetches it using a utility.
- Auto-saving is temporarily disabled by default to avoid saving immediately.
- It then grabs the save interval (e.g., 60 seconds) from the save settings.
- It also loads the last-used autosave slot from PlayerPrefs, allowing saves to continue cycling smoothly even after restarting the game.

2. Timing – FixedUpdate()

This method is called on a fixed interval by Unity (ideal for time-based operations):

- It adds time to the timeSinceLastSave.
- Once the accumulated time passes the defined saveInterval, it checks if automatic saving via events is turned off.
- If so, it triggers the autosave using SaveAutoGame().

This approach ensures that saving occurs in the background, without depending on player actions.

3. Saving – SaveAutoGame()

This is the core autosave method:

- It first checks whether autosaving is enabled in the settings (autosaveEnabled must be true).
- If it is, it sets the filename based on the current slot, such as "0 - 3".
- It tells the SaveCustomInScene component to perform the save.
- It then increments the save slot, cycling from 1 to 6 in a loop.
- The updated slot is stored in PlayerPrefs to persist between sessions.

Usage Scenario

You would typically attach this component to a game manager or an invisible object in the scene. Once the game starts, it will:

- Automatically begin a save cycle using the configured interval (e.g., every 60 seconds),
- Skip saves if autosaving is turned off or if an event-based save is active,
- Save to slots in sequence (slot 1, then 2, up to 6, then back to 1),
- Ensure the last-used slot is remembered between play sessions.

This setup is ideal for games that require **fail-safe saving in the background**, such as open-world or survival games where frequent progress recording is essential.

SaveCustomInitialization

This script handles the **automatic setup** of a custom game save system when the game first launches. It ensures that the saving system is initialized **only once** and stays active even when the player moves between scenes. It's a key part of creating a centralized, persistent save system that works in the background without needing manual configuration in each scene.

Purpose

The goal of this script is to:

- Load the core save configuration (SaveCustomObject) from the Unity Resources folder.
- Create a GameObject in the scene that will manage saving.
- Attach two saving components to it: one for actual saving logic and one for autosaving at intervals.
- Make this GameObject **persistent**, so it is not destroyed when changing scenes.

Class: SaveCustomInitialization

This class is **not a MonoBehaviour**. It doesn't need to be added to any GameObject. It runs automatically at game startup thanks to a special runtime attribute.

Variables and Their Roles

saveCustomObject

- A **static** reference to the save configuration object.
- Holds core save settings like slot information, autosave toggle, intervals, etc.
- Loaded from Unity's **Resources** system using a specific filename.

Main Method: RunGameInitialization()

This method is marked with a runtime attribute that tells Unity to **automatically execute it when the game starts**, even before the first scene is loaded.

What it does:

1. Logs a message to confirm the save system is initializing.
2. Calls LoadSettingsData() to fetch the main save configuration.
3. If the configuration is missing, it logs an error and stops further setup.
4. If successful:
 - Creates a new GameObject named [Save Custom Object].
 - Adds two important components to it:
 - SaveCustomInScene: handles manual saving operations.
 - AutoSaveCustom: handles timed automatic saving.
 - Links the references between components:
 - The scene saver gets the configuration object.
 - The auto-saver gets the scene saver component.
 - Makes the GameObject **persistent** using Unity's "Don't Destroy On Load," so it won't disappear when loading a new scene.

This setup ensures the save system runs **invisibly and globally**, no matter how many scenes the game has.

Secondary Method: LoadSettingsData()

This method attempts to load a SaveCustomObject from Unity's **Resources folder** using the expected filename: "Save Custom Object Data".

If it fails:

- It logs an error, alerting the developer that something is wrong with the project setup — perhaps the object was deleted or not placed in the correct folder.

Usage Scenario

This class doesn't need to be added to the scene or manually triggered.

Instead:

- As soon as the game launches, Unity calls RunGameInitialization().
- The save system is immediately ready and running.
- From this point on, any save/load calls in your game will use the centralized and persistent GameObject created here.

This is perfect for **large games with multiple scenes**, since the save system won't need to be manually duplicated or reloaded — it just works globally from the start.

SaveCustomInScene

This script is the **core logic** for managing save and load operations that are specific to the current game scene. It tracks time played, updates save data regularly, captures screenshots, and can write or read save data from **disk or PlayerPrefs** depending on how the system is configured.

It's intended to work alongside the auto-save system but can also be triggered manually (e.g., from a save menu). Everything runs on a central component attached to a persistent GameObject.



Main Class: SaveCustomInScene

This is a component-based class, meant to be attached to a GameObject (automatically by the initialization script). It controls saving the current scene state and restoring it later.



Key Responsibilities

- Track how much in-game time has passed.
- Keep the current scene name updated.
- Save a screenshot for visual feedback in the save menu.
- Serialize all relevant data into JSON and save it.
- Load saved data and restore the scene and game state.



Variables Overview

saveCustomObject

Holds the master configuration and data container for the save system. Includes things like the list of items to save, the current scene name, time played, screenshot, and storage preferences.

gameTime

A formatted string showing how long the player has been in the current session (e.g., "00:42:09").

fileName

The base filename used when saving to disk or PlayerPrefs.

savePath

The full path on disk where the save file is stored when not using PlayerPrefs.

sceneName

The name of the currently active scene, used to track and reload the same scene on load.

sceneCamera

The camera used to take a screenshot for save thumbnails. It is detected automatically based on the player object or fallback options.

elapsedTime

How many seconds have passed since the session started. Used to compute the in-game time string.



Methods Breakdown

ResetSave()

Clears temporary data in saveCustomObject and resets the timer. It does **not delete** the save file—just clears fields related to the current session like the screenshot, time, and scene name.

FixedUpdate()

Runs at regular intervals (independent of frame rate). It:

- Updates the elapsedTime.
- Recomputes the gameTime string.
- Assigns the current time and scene name to saveCustomObject.

This ensures the save data stays up to date with what's happening in-game.

UpdateSaveCustomObject()

Sets the current scene name and game time in saveCustomObject, keeping the data in sync.

UpdateGameTime()

Takes the float elapsedTime and converts it to a formatted string like 01:23:45 for display and saving.

GetCamera()

Automatically detects the correct camera to use for taking screenshots:

1. Looks for a GameObject tagged "Player" and checks for a camera on it or its children.
2. Falls back to the main camera.
3. As a last resort, finds any camera in the scene.
4. Logs an error if no cameras are found.

This lets the system adapt to different scene setups without manual assignment.

SaveData()

This is the big one:

1. Makes sure there's a camera available.
2. Captures a screenshot and stores it in `saveCustomObject`.
3. Builds a `SaveCustomFile` object with all current game data.
4. Converts it to a JSON string.
5. Saves it either:
 - To **PlayerPrefs** (if enabled), or
 - To a file on disk:
 - Under `persistentDataPath` for runtime builds.
 - Under `Assets/Editor` in the Unity editor.

It also handles creating directories if needed and logs where it saved the file.

LoadData()

Restores previously saved data:

- Checks whether the system is set to use `PlayerPrefs` or file saving.
- Reads the save data string.
- Converts it back into a structured object.
- Passes it to `LoadDataString()` to apply it.

LoadDataString(data)

Takes a deserialized save object and applies it to the current state:

- Restores time, scene, and item data.
- Loads the correct scene (restarting it).
- Rebuilds the save item list.

This is what makes actual "loading" happen — whether coming from a file or `PlayerPrefs`.

Class: SaveCustomFile

This class defines what is saved. It's a serializable container for all relevant data, including:

- Screenshot as a byte array.
- In-game time string.
- Scene name.
- List of custom saveable items.
- Elapsed time as a float.

This class is used for both saving and loading — it holds all the state we care about.

Usage Flow

On Save:

1. Game calls `SaveData()` (e.g., through a save menu or autosave).
2. Camera captures a screenshot.
3. Data is packaged into a structured object and saved as JSON.
4. File is written to disk or stored in `PlayerPrefs`.

On Load:

1. Game calls `LoadData()` (e.g., from the load menu).
2. JSON data is read and deserialized.
3. Scene is loaded and data applied.

SaveCustomObject

Overview

This script defines a **data storage system** for custom game saves. It uses a **ScriptableObject** to hold structured save information such as screenshots, scene names, game time, and customizable user-defined values (like vectors, floats, integers, strings, and booleans). Additionally, it includes Unity Editor functionality to create this object as an asset.

Editor Utility (Only in Unity Editor)

This section only runs in the Unity Editor and doesn't get included in the build.

Menu Item: Create SaveCustomObject Asset

- Adds a menu option in Unity to create a new custom save data asset.
- It stores the asset in the `Assets/Resources` folder.

- If an asset already exists there, it prompts the user with a confirmation dialog to overwrite it.
- After creation, it selects the asset in the Project window for easy configuration.

ScriptableObject Class: SaveCustomObject

This is the core data container used for saving custom game data.

Screenshot Storage

- Stores a screenshot as raw byte data.
- Used for saving a thumbnail of the scene when the game is saved.

Pixel Limit

- Sets the maximum resolution for screenshot captures to keep file sizes manageable.

Game Time

- Holds the current in-game clock as a string (formatted as hours:minutes:seconds).

Scene Name

- Holds the name of the scene where the game was saved.

Auto-Save Settings

These variables control whether auto-saving is enabled and how it's triggered.

- **Enable Auto Save:** Master switch to turn auto-saving on or off.
- **Save by Event:** Triggers auto-saving based on custom game events (like checkpoints).
- **Save by Time:** Triggers auto-saving based on a repeating time interval.
- **Save Interval:** Sets how often to save when time-based auto-save is enabled.

Saving Mode Options

These booleans define where and how the data is saved:

- **Game Data:** Indicates if the object contains actual game data.
- **Local Low:** Saves the file in the system's persistent data path (safe and platform-independent).
- **Player Prefs:** Saves using Unity's PlayerPrefs system (not file-based, useful for lighter or faster saves).

Custom Items

This is the most flexible part. It stores a **list of items**, where each item is a package containing multiple data types that can be customized and saved:

SaveCustomItem Class

Each item in the list has:

- A **tag** for identifying what the item is (like a variable name).
- Multiple lists, each for a different data type:
 - **Vectors** (4D)
 - **Floats**
 - **Integers**
 - **Strings**
 - **Booleans**

Each of these lists can hold several values, each with its own tag and data.

Custom Value Types

Each of the following classes stores one specific data type with a label:

SaveCustomVector

- Holds a 4D vector (x, y, z, w) and a descriptive tag.

SaveCustomFloat

- Holds a single float value and a tag.

SaveCustomInt

- Stores an integer with an identifier.

SaveCustomString

- Contains a string value with a tag.

SaveCustomBool

- Stores a boolean (true/false) with a tag.

Usage Example

- This ScriptableObject is designed to be used by the **SaveCustomInScene** script (or similar ones).

- It allows developers or designers to **track and persist a wide range of data** between play sessions.
- Thanks to its flexibility, it can be used for player stats, settings, inventory, world states, and more.

CheckButtonSelected

 **Purpose of the Script**
This script is designed to ensure smooth **keyboard or controller navigation** in Unity's UI system. It automatically re-selects a default button whenever no UI element is selected or if the currently selected UI element becomes inactive.

This is especially useful in menus where:

- Input is handled through a gamepad or keyboard (not a mouse).
- Unity's EventSystem might lose focus due to scene changes or UI transitions.

 **Component Metadata**

- The script is marked to appear in the Unity component menu under UI > Save Custom Game > Check Button Selected.
- This makes it easy to add it to a GameObject from the Unity Editor.

 **Serialized Field: Default Button**
This is a reference to a button in the UI which acts as the **fallback option**. If nothing else is selected or the currently selected object is inactive, this is the button that will be selected.
This button must be assigned manually in the Unity Inspector.

 **Update Method (Runs Every Frame)**
Every frame, the script performs the following logic:

1. **Checks if there is any currently selected UI object:**
 - If nothing is selected, that means the EventSystem has no focus, which can break keyboard/controller navigation.
2. **Checks if the currently selected object is active in the hierarchy:**
 - Sometimes an element may technically be selected but has been hidden or deactivated in the UI. This would also cause input issues.
3. **If either of the above checks fails:**
 - It **automatically selects** the default button specified earlier.
 - This restores a valid selection state, ensuring input navigation continues smoothly.

 **Typical Usage Scenario**
This script would be used in menus like:

- Pause menus
- Main menus
- Save/load menus
- Any UI where navigation is critical

It's especially important in gamepad/controller-supported games, where users **can't use a cursor** to reselect UI manually.

 **Setup Tips**
To make it work properly:

- Make sure the UI uses Unity's EventSystem (usually auto-added when you create a Canvas).
- Assign a valid button to the default button field.
- Place this script on any persistent UI controller object (like a menu manager or UI root).

DefaultSelectedButton

 **Purpose of the Script**
This script ensures a **specific button is automatically selected** whenever the GameObject it's attached to becomes **active** in the scene.
Its main role is to provide **consistent UI navigation**, particularly for users using a **keyboard or gamepad**, by resetting which UI element is selected as soon as a menu or panel becomes visible.

Serialized Field: Button

- This is a reference to the **default button** that should be selected when the GameObject is enabled.
- It's configured in the Unity Inspector.
- The button acts as the entry point for navigation when the UI becomes visible again.

OnEnable Method (Triggered When the Object is Activated)

This method automatically runs when the GameObject the script is attached to becomes active (e.g., when a menu panel is opened).

It performs two simple but essential actions:

1. **Clears the current selection** in Unity's EventSystem:
 - This step removes focus from any previously selected UI element.
 - It prevents the EventSystem from trying to retain selection across menus, which can cause broken navigation or unexpected UI behavior.
2. **Selects the predefined button:**
 - This action restores focus to a known, valid UI element.
 - It guarantees that when the player opens the menu or UI screen, input immediately begins at the correct spot.

When and Why to Use It

This is ideal for any situation where:

- A **menu** or **UI panel** appears.
- You want to **force navigation to begin at a specific button** (such as "Continue", "New Game", or "Back").
- You're relying on non-mouse input methods like gamepads or keyboard.

Examples:

- A pause menu where the "Resume" button is always highlighted first.
- A save/load menu where the first slot is auto-selected.
- A confirmation dialog where the default action is pre-highlighted.

Setup and Usage Tips

- Add the script to any UI GameObject that becomes active at runtime (e.g., menus, panels).
- Drag a UI Button into the button reference field in the Inspector.
- Make sure your UI system includes Unity's **EventSystem**, which handles input focus.

LoadMenuManagerTMP and LoadMenuManager

The LoadMenuManager script is responsible for managing a user interface that allows players to load saved game data through a multi-slot menu system.

Overview and Purpose

This script enables players to load saved game states either from PlayerPrefs or from local .json files. It presents multiple save slots (six per page), allows page navigation, shows thumbnails of each save, and handles confirmation before loading a game. It also displays an appropriate title depending on whether the selected save is an autosave or a manual save.

Serialized Fields (Inspector Configurations)

Load Settings

- **saveCustomInScene**: The core component that performs the actual data loading.

Confirmation Panel

- **confirmationPanel**: UI panel that shows up before confirming the load.
- **confirmButton** / **cancelButton**: Buttons to either proceed or cancel the load.

Title System

- **titleLoad**: UI text that shows the current slot's title ("Load" or "Autosave").
- **text**: Default title for regular save slots.
- **textAutomatic**: Display name for the autosave slot (slot 0).

Button System (For Slots 1 to 6)

Each slot has:

- A **string** for the load path.
- A **Button** for interaction.

- A **RawImage** for a thumbnail or preview.
- A **Text** for the label.

Page System

- **right / left**: Buttons to move forward or backward through the save pages.
- **inputField**: Accepts manual input of the page number.



Internal State

- **currentLoadNumber**: Tracks which page (or save group) the user is currently viewing.
- **firstTime**: Ensures that button setup runs only once.
- **loadKey**: A key used in PlayerPrefs to remember the user's last accessed save group.
- **OnDisable**: Automatically clears all thumbnail previews when the UI is disabled.



Lifecycle and Behavior

OnEnable

When the script is enabled, it:

- Fetches the **SaveCustomInScene** reference.
- Loads the last used save group from **PlayerPrefs**.
- Sets up the UI buttons and input field (once).
- Sets the correct title and slot labels.

OnDisable:

- Clears all preview textures when the load menu is closed.



UI Initialization and Interaction

SetupButtonsAndInputField

- Binds listeners to navigation buttons.
- Binds validation to the input field.
- Binds load actions to each of the six load buttons.
- Binds cancellation logic to the cancel button.

SetupInitialLoadName

- Updates the input field text and refreshes the UI with slot names and data.



Logic for Navigation and Slot Display

SetTitle

- Changes the title label between "Autosave" (if slot 0) or "Load".

IncreaseNumber / DecreaseNumber

- Increment or decrement the current save group index and refresh the UI.

ValidateInput

- Ensures only numbers are typed into the input field.

OnEndEditInputField

- Applies the number the user typed to **currentLoadNumber** and refreshes the UI.



Slot Management

UpdateLoadNames

- Updates all six slot labels with a prefix showing the current page and slot number.
- Enables/disables navigation buttons.
- Clears existing previews.
- Loads metadata for each slot.
- Updates the title.

LoadData

- For each slot:
 - Generates the expected file name.
 - Determines where to load it from (**PlayerPrefs** or file).
 - Loads the save data and displays a preview or disables the slot if empty.

DetermineLoadPath

- Builds the path to where a save file is located, depending on whether **PlayerPrefs**, **LocalLow**, or **Editor** paths are used.



Slot Interaction

LoadGame

- Triggers confirmation only if the slot has data.

ConfirmLoad

- Sets the filename and path on the save loader.
- Clears preview images using **ClearRawImagesAndTextures**.
- Loads the actual game.

CancelLoad

- Closes the confirmation panel without doing anything.

IsLoadSlotOccupied

- Checks whether a specific slot has a preview image (indicating presence of data).

ClearRawImagesAndTextures

- Resets all preview images.
- Destroys existing texture memory to prevent leaks.

InteractableSlot

- Toggles visibility and interactivity of a slot based on whether data is found.



Usage

- This script is typically placed on a UI manager GameObject in a Load Game scene.
- The developer connects buttons, text fields, and preview slots in the Unity Inspector.
- The player navigates through pages of save slots, selects one, and clicks to load.
- If the selected slot has data, a confirmation prompt appears.
- Once confirmed, the game state is loaded via the SaveCustomInScene component.

SaveMenuManagerTMP and SaveMenuManager

The **SaveMenuManager** script is responsible for managing a save system UI in a Unity game. It provides a structured interface for saving game states across multiple slots, with support for PlayerPrefs and file-based saving.



Purpose and General Use

This script allows the player to save their game into one of several slots through a graphical interface. The system supports up to 6 slots per page and allows navigation between pages using input fields and buttons. It handles both **saving** and **UI updates**, offering visual thumbnails and slot titles. It also prevents accidental overwrites with a confirmation panel.



Serialized Fields and Variables



Save Settings

- **SaveCustomInScene (saveCustomInScene)**: Reference to the component that performs the actual game data save logic.



Confirmation Panel

- **confirmationPanel**: A popup that appears to confirm overwriting an existing save.
- **confirmButton / cancelButton**: Buttons inside the confirmation panel used to confirm or cancel saving.



Title System

- **titleLoad**: The UI text showing the current page's save name or "Autosave".
- **text / textAutomatic**: Strings to customize the save slot titles ("Save", "Autosave").



Button System (Per Slot)

For each of the 6 save slots:

- **Button (e.g., buttonSave1)**: Button for selecting that slot.
- **RawImage (e.g., rawImageSave1)**: Preview image thumbnail.
- **Text (e.g., textSave1)**: Text displaying the slot number and page.



Page System

- **right / left**: Buttons to navigate between save pages.
- **inputField**: Allows manual entry of the save page number.



Other Internals

- **currentSaveNumber**: Tracks the current page of save slots.
- **firstTime**: Ensures some setup code only runs once.

- **saveKey:** String used as the PlayerPrefs key to remember the last selected page.

Method Summaries and Logic

OnEnable

Called when the UI opens. Sets up components, restores last save page number from PlayerPrefs, initializes UI components and listeners.

OnDisable

Called when the UI is disabled. Clears the raw images and textures from the save slots to prepare for the next session.

SetTitle

Updates the top title label based on the current page:

- If on page 0, shows "Autosave".
- Otherwise, shows standard save title.

SetupButtonsAndInputField

Configures the buttons and input field:

- Sets up listeners for navigation and slot buttons.
- Links confirm and cancel buttons to their respective methods.
- Prevents reinitialization if already done once.

SetupInitialSaveName

Initializes the input field text with the current page number and refreshes the slot labels.

UpdateSaveNames

- Updates each slot label (e.g., "3 - 1") to reflect the current page.
- Stores current page to PlayerPrefs.
- Enables/disables navigation buttons appropriately.
- Clears old previews and loads updated data.
- Updates the UI title.

IncreaseNumber / $\equiv$ DecreaseNumber

Increments or decrements the page number:

- Updates the input field and calls UpdateSaveNames to reflect the change.

ValidateInput

Restricts user input in the field to numeric digits only.

OnEndEditInputField

Processes the number after editing ends in the input field. Parses the value and updates the page and slot data.

SaveGame

Called when a slot button is clicked:

- If the slot has existing data, opens the confirmation panel.
- Otherwise, constructs a save file name and triggers the actual save logic.
- Reloads UI to show the new save data.

IsSaveSlotOccupied

Checks whether a slot already contains data (i.e., has a preview image assigned).

ClearRawImagesAndTextures

Removes all preview textures to prepare for loading new data.

✅ **ConfirmSave**

Runs when the user confirms overwriting:

- Saves the game data using the constructed file name.
- Reloads data and closes the confirmation dialog after the save is complete.

❌ **CancelSave**

Closes the confirmation panel without doing anything.

💾 **LoadData**

Iterates through all 6 slots and loads data:

- Uses PlayerPrefs or file system depending on user settings.
- Loads and deserializes data.
- Updates preview images and visibility based on presence of data.

📁 **DetermineSavePath**

Generates the correct file path for the save file:

- Adjusts for PlayerPrefs, Application.persistentDataPath, and whether it's in Editor or build mode.

🎨 **RenderImageSlot**

Controls whether the preview image is shown or hidden, depending on whether that slot is used.

📄 **LoadDataString (Assumed External)**

While not shown in your snippet, this method would populate UI elements with the save file's information, such as game state or metadata.

🧑💻 **Usage Example**

1. When the user opens the save menu, the UI loads the last used page.
2. The user navigates using the arrows or types a page number.
3. Each page shows 6 save slots labeled with their full names (e.g., "2 - 3").
4. If the user clicks a slot that has data, a confirmation prompt appears.
5. If confirmed, the data is saved and a preview is shown.
6. If a slot is empty, the game is saved immediately.
7. Previews update automatically based on the save format (PlayerPrefs or file-based).